AttestLine

Cross-chain credit lines on Creditcoin, underwritten by attested on-chain activity.

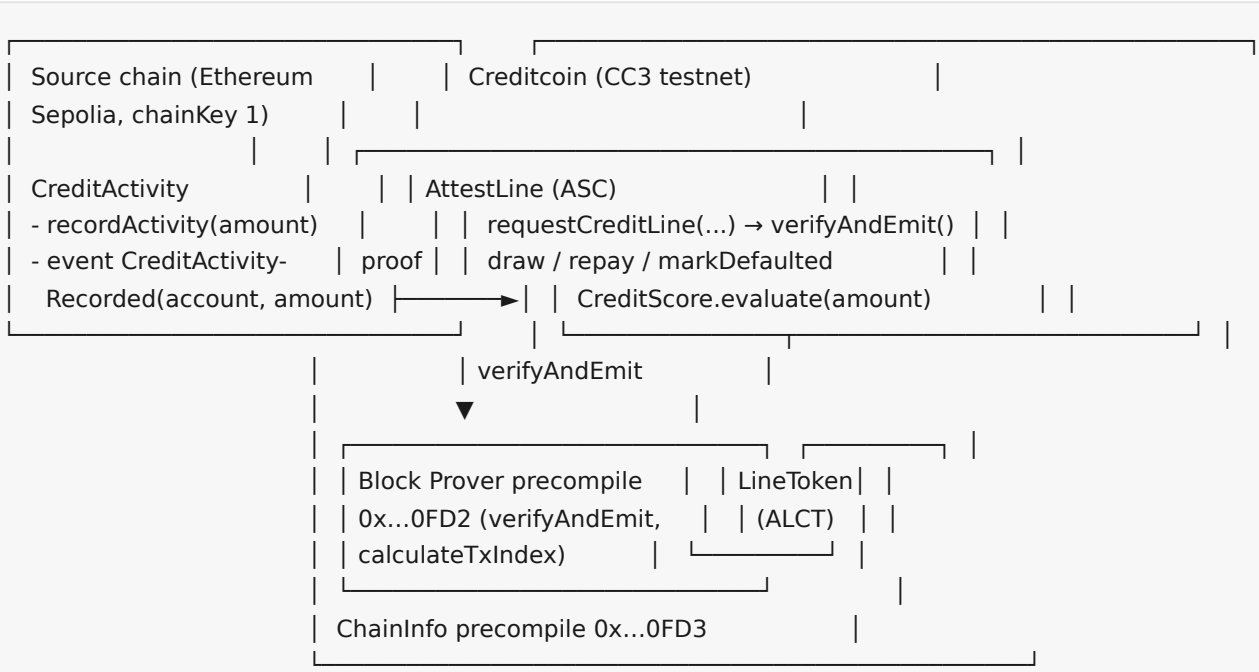
AttestLine is a DeFi protocol on Creditcoin that grants on-chain credit lines to borrowers based on attested cross-chain activity, verified natively through the Attestcoin Protocol's Block Prover precompile — no oracle operators, no off-chain credit bureaus.

This document bundles the architecture overview and the Attestcoin Protocol integration deep dive.

AttestLine — Architecture

This document describes the design of AttestLine, the exact formulas used, the security model, and the decisions taken. It complements docs/attestcoin-integration.md, which focuses on the Attestcoin Protocol integration itself.

1. System overview



The borrower:

1. commits value/activity on the source chain (`recordActivity`),
2. proves that transaction to `AttestLine` (Merkle + continuity proof),
3. is underwritten deterministically (score → limit → APR),
4. draws `LineToken` (minted by the protocol) up to her limit,
5. repays principal + interest.

Everything is deterministic and on-chain. The proof comes from the Attestcoin Protocol block prover: no centralized oracle operators.

2. Contracts

2.1 `contracts/source/CreditActivity.sol` (source chain)

Minimal activity ledger, per Attestcoin best practices:

- a single source contract,
- one unambiguous event that carries all data: `CreditActivityRecorded(address indexed account, uint256 amount)` ,

- no hidden state transitions (cumulative `activityOf` view only).

The event name is unique to this dApp so `AttestLine` can filter for it unambiguously after verifying inclusion.

2.2 `contracts/AttestLine.sol` (Creditcoin ASC)

The core contract. Key state:

State	Purpose
`VERIFIER` (immutable)	Block Prover precompile instance from `NativeQueryVerifierLib.getVerifier()` (`0x...0FD2`)
`processedQueries`	Replay protection: one proof key per (chainKey, blockHeight, txIndex)
`sourceContract`	The ONLY source-chain contract whose activity events are accepted (owner-settable)
`lineToken`	Token the protocol lends (owner-settable once)
`creditLines`	`CreditLine { borrower, creditLimit, used, interestRateBps, createdAtBlock, deadlineBlock, defaulted }`
`_interest`	Per-borrower `{ accruedInterest, lastAccrualBlock }` (separate mapping; see §4)
`lastActivityAmount`	Amount from the borrower's most recent accepted proof
`termBlocks`	Term of a new line (constructor parameter; default 648,000 ≈ 90 days at 12s blocks)

`requestCreditLine` pipeline:

1. **Replay protection** — derive `txIndex` from the Merkle sibling path via `VERIFIER.calculateTxIndex(merkleProof)` ; compute `txKey = keccak256(chainKey, blockHeight, txIndex)` using the same packed layout as the official ASCMinter example; revert `ProofAlreadyProcessed` if seen.
2. **Verify inclusion** — build `MerkleProof { root, siblings }` and `ContinuityProof { lowerEndpointDigest, roots }` ; call `VERIFIER.verifyAndEmit(...)` (state-changing, emits `TransactionVerified` on the precompile); revert on failure.
3. **Mark processed**.
4. **Decode + validate** — `EvmV1Decoder.getTransactionType` must be valid; `decodeReceiptFields(...).receiptStatus == 1` (the precompile proves inclusion only, never success).
5. **Extract the event** — `getLogsByEventSignature(receipt, CREDIT_ACTIVITY_EVENT_SIGNATURE)` must return ≥ 1 log; the log's emitting address (`log.address_`) must equal `sourceContract` .
6. **Bind to the caller** — the indexed `account` must equal `msg.sender` ; the amount must be > 0.
7. **Underwrite** — `CreditScore.evaluate(amount)` → store the line (`deadlineBlock = block.number + termBlocks`), reset interest state, emit `CreditLineGranted` .

One-line-per-borrower rule. A borrower may hold at most one credit line. A new (or refreshed) line is only granted when the previous line has **no outstanding principal** (`used == 0`).

Attempting to re-grant with outstanding debt reverts `ActiveCreditLineExists`. Rationale: keeping the accounting single-line makes interest and default handling unambiguous, and mirrors how credit lines behave in traditional finance (one facility at a time). A defaulted line that has been fully repaid does not permanently bar re-application, but the `defaulted` flag remains visible as a permanent reputation record.

Lending functions:

- `draw(amount)` — requires an active (non-defaulted, before-deadline) line and sufficient remaining limit; accrues interest; `used += amount`; `lineToken.mint(msg.sender, amount)`.
- `repay(amount)` — see the interest model (§4). Requires `amount ≥ accrued interest`; interest is settled first, the remainder reduces principal; overpayment reverts; `used == 0` → `LineSettled`.
- `markDefaulted(borrower)` — anyone may call once `block.number > deadlineBlock` and `used > 0`; accrues and freezes interest, sets `defaulted = true` (draws frozen, repayment still allowed).

2.3 `contracts/CreditScore.sol` (pure library)

Deterministic function of the attested amount only — no storage, no external calls. See §3.

2.4 `contracts/LineToken.sol`

`LineToken` ("AttestLine Credit Token", symbol `ALCT`) — OpenZeppelin ERC20 with `mint` / `burn` restricted to the `minter` (the AttestLine ASC, set at construction). It is a plain protocol credit token with no transfer restrictions; it has no intrinsic value in v1 and is fully governed by the protocol's underwriting (reputation) model.

2.5 `contracts/mocks/MockVerifier.sol` + `TestHarness.sol` (test only)

`MockVerifier` is a faithful model of the Block Prover precompile's observable behavior:

- `calculateTxIndex` derives the transaction index from the Merkle sibling path (LSB-first: `sibling[i].isLeft` sets bit `i`),
- `verify` / `verifyAndEmit` recompute the block root with the canonical Keccak scheme (`leaf = keccak256(0x00 ‖ tx)`, `inner = keccak256(0x01 ‖ left ‖ right)`), enforce an expected (chainKey, height) pair in strict mode, and revert on mismatch — like the Rust precompile.

Tests inject its bytecode at `0x...0FD2` via `hardhat_setCode` + `hardhat_setStorageAt`, so AttestLine's real code path — calling the precompile address through `NativeQueryVerifierLib.getVerifier()` — is exercised deterministically. `TestHarness` lets a contract be the borrower and draw+repay within a single block (zero elapsed blocks → zero interest), mirroring batched repayments and letting tests cover the full lifecycle without needing to source extra ALCT for interest.

3. Credit scoring model

`CreditScore.evaluate(attestedAmount)` returns `(score, limitFactorBps, aprBps)`:

```
tokens = attestedAmount / 1e18          // whole tokens of committed value
steps  = floor(log2(tokens + 1))        // logarithmic "activity band"
score  = min(850, 300 + 50 · steps)     // FICO-like 300–850 scale
```

steps	tokens range	score	limit factor (× attested)	APR
0	0	300	0.5× (5 000 bps)	18% (1 800 bps)
1	1	350	0.8× (8 000 bps)	15% (1 500 bps)
2	2–3	400	1.0× (10 000 bps)	12% (1 200 bps)
3	4–7	450	1.2× (12 000 bps)	10% (1 000 bps)
≥ 4	≥ 8	500–850	1.5× (15 000 bps)	8% (800 bps)

`creditLimit = attestedAmount · limitFactorBps / 10_000`.

Properties:

- **Deterministic** — pure function of the attested amount; trivially unit-testable.
- **Monotone** — score and limit factor never decrease with amount; APR never increases.
- **Saturating** — score caps at 850 (≥ 2 047 tokens); the floor is 300 (amount 0 — though AttestLine rejects zero-amount grants).
- The logarithm compresses large differences: doubling activity adds a fixed score increment, so a borrower cannot meaningfully game the score by over-committing trivially more value.

4. Interest model

Linear per-block interest, interest-first settlement.

- `interest = principal · rateBps · elapsedBlocks / (10_000 · BLOCKS_PER_YEAR)` with `BLOCKS_PER_YEAR = 2_102_400` (Creditcoin produces a block roughly every 12 seconds).
- Interest accrues lazily: `_accrueInterest` is called on every interaction and in `accruedInterestOf`; the per-borrower `lastAccrualBlock` is advanced to the current block, so elapsed is exact and no one can game the rounding.
- `accruedInterest` starts accruing at the **draw block** (the `lastAccrualBlock` is reset on draw), not the grant block.
- **Every repayment must first settle ALL interest accrued to date**; the remainder (if any) reduces principal. Repayments that don't cover accrued interest revert (`RepayDoesNotCoverInterest`); overpayments revert (`Overpay`). When `used` reaches zero the line is settled (`LineSettled`) and a new line can be requested.
- Interest paid stays in the protocol (AttestLine holds the tokens) — v1 treats it as protocol revenue; there is no borrower-facing treasury contract yet.
- **Defaults freeze interest**: `markDefaulted` accrues up to the default block and the accrual clock stops; `accruedInterestOf` returns the frozen value afterwards.

Why interest-first? It keeps accounting exact and simple: `used` is always pure principal, and a borrower can never "repay the principal and strand the interest". The model is fully deterministic and covered by exact-equality tests in `test/AttestLine.test.ts`.

5. Replay protection & proof keys

- `txIndex` is derived from the Merkle sibling path via `VERIFIER.calculateTxIndex(merkleProof)` (LSB-first: sibling `i` being `isLeft` sets bit `i`).
- `txKey = keccak256(chainKey, blockHeight, txIndex)` — computed with the exact packed layout used by the official ASCMinter example (32-byte chainKey word, blockHeight in the top 8 bytes of the next word, txIndex in the following 8 bytes, 72 bytes hashed).
- `processedQueries[txKey] = true` is set **after** successful verification and the key is never reused, so a proof cannot be replayed (neither by the borrower to re-grant nor by anyone else).

6. Security model

- **Verification is native.** The precompile verifies Merkle inclusion + block continuity against the Creditcoin-attested header chain. AttestLine never trusts a submitted root on its own.
- **Inclusion ≠ success.** The receipt status check (`== 1`) is mandatory — the precompile only proves the transaction was included.
- **Source-contract allowlist.** Only logs whose emitting address equals the owner-registered `sourceContract` are accepted (mirrors the official `USCLoanManager`), preventing anyone from deploying a contract that emits a matching event.
- **Account binding.** The attested account must equal `msg.sender` , so no one can underwrite a line on someone else's attested activity.
- **Reentrancy.** All external functions are `nonReentrant` (OpenZeppelin) and follow checks-effects-interactions.
- **Ownership.** OpenZeppelin `Ownable` gates `setSourceContract` / `setLineToken` .
- **Reputation-collateralized.** v1 has no liquidation of collateral. Defaulting is permissionless after the deadline, freezes draws, and is a permanent reputation record. This is a deliberate product decision: the "collateral" is the borrower's on-chain credit history.

7. TypeScript layer

- `src/config.ts` — env-var configuration with CC3 testnet defaults and validation; never logs keys.
- `src/client.ts` — `AttestLineClient` wraps `@gluwa/usc-sdk` (`proofProvider.service.ProofBuilder` , `chainInfo.PrecompileChainInfoProvider`) and the TypeChain contracts:
  - `buildProof(txHash)` — waits for attestation (`waitUntilHeightAttested` , 15s poll / 15m timeout), fetches the proof, decodes the `CreditActivityRecorded` event from the EvmV1-encoded transaction,
  - `requestCreditLine(txHash, signer)` — fast pre-validates the source receipt (status, emitter, account) before waiting for attestation, then submits to the ASC,

    - `draw` / `repay` / `getCreditLine` / `creditScoreOf` / `available` / `accruedInterestOf`,
    - `getSupportedChains()` — ChainInfo precompile.
- `src/types.ts` — shared types.

8. Deviations & decisions

Topic	Decision
`INativeQueryVerifier`	Vendored in `contracts/interfaces/` because the `@gluwa/usc-contracts` npm copy omits `verifyAndEmit` / `calculateTxIndex`; shapes are byte-identical with the official example + precompile ABI.
Library linking	Hardhat does not auto-link libraries (including node_modules ones); deploy scripts/tests link `EvmV1Decoder` and `CreditScore` explicitly.
`termBlocks`	Constructor parameter (default 648,000 ≈ 90 days) instead of a hard constant, so deadline tests can mine past it cheaply.
Interest state	`InterestState { accruedInterest, lastAccrualBlock }` kept in a separate mapping to preserve the exact `CreditLine` struct shape from the spec.
`_interest` visibility	`internal` mapping with public views (`accruedInterestOf`) instead of a public mapping, for a cleaner API.
One line per borrower	Re-grant requires `used == 0`; defaults are permanent reputation records but don't bar re-application once fully repaid.
`accruedInterestOf` / `available`	Added as views beyond the spec's minimum for testability and UX.
Draw deadline	`draw` also requires `block.number <= deadlineBlock` (`LineExpired`) — draws after the term end make no sense.
Test harness	`TestHarness` (test-only) batches draw+repay in one block to exercise the full lifecycle deterministically without external ALCT.
Build	`npm run build` = `tsc -p tsconfig.build.json` (emits CommonJS `dist/` for the SDK layer); `npm run typecheck` = `tsc --noEmit` over the whole repo.
Worker	Because AttestLine requires `account == msg.sender`, the worker can only submit proofs for accounts whose keys it holds (`WORKER_PRIVATE_KEYS`).

AttestLine × Attestcoin Protocol — integration deep dive

AttestLine is an **Attestcoin Smart Contract (ASC)**: a Creditcoin smart contract whose underwriting inputs are *native inclusion proofs* of transactions that happened on another chain. This document explains exactly how that integration works — the interfaces, the proof flow, the encoding, and the security pitfalls the code guards against.

1. The Attestcoin Protocol in one paragraph

The Attestcoin Protocol (Gluwa) lets Creditcoin smart contracts *read* what happened on other EVM chains without trusting an oracle. Validators attest to source-chain block headers; the **Block Prover precompile** on Creditcoin stores the resulting attestation chain. A dApp submits a transaction it cares about, ABI-encoded in the canonical "EvmV1" format, along with:

- a **Merkle proof** (block root + sibling path) proving the transaction is in a specific block, and
- a **continuity proof** (a chain of block-root digests) proving that block descends from an attested header.

The precompile verifies both natively against the attestation chain and, on success, the dApp can *decode* the transaction and its receipt and act on it.

2. Precompiles & addresses

Precompile	Address	Used for
Block Prover (`INativeQueryVerifier`)	`0x0000000000000000000000000000000000000FD2`	`verify` / `verifyAndEmit` / `calculateTxIndex`
ChainInfo	`0x0000000000000000000000000000000000000FD3`	chain list, attestation heights/bounds (via `@gluwa/usc-sdk`)

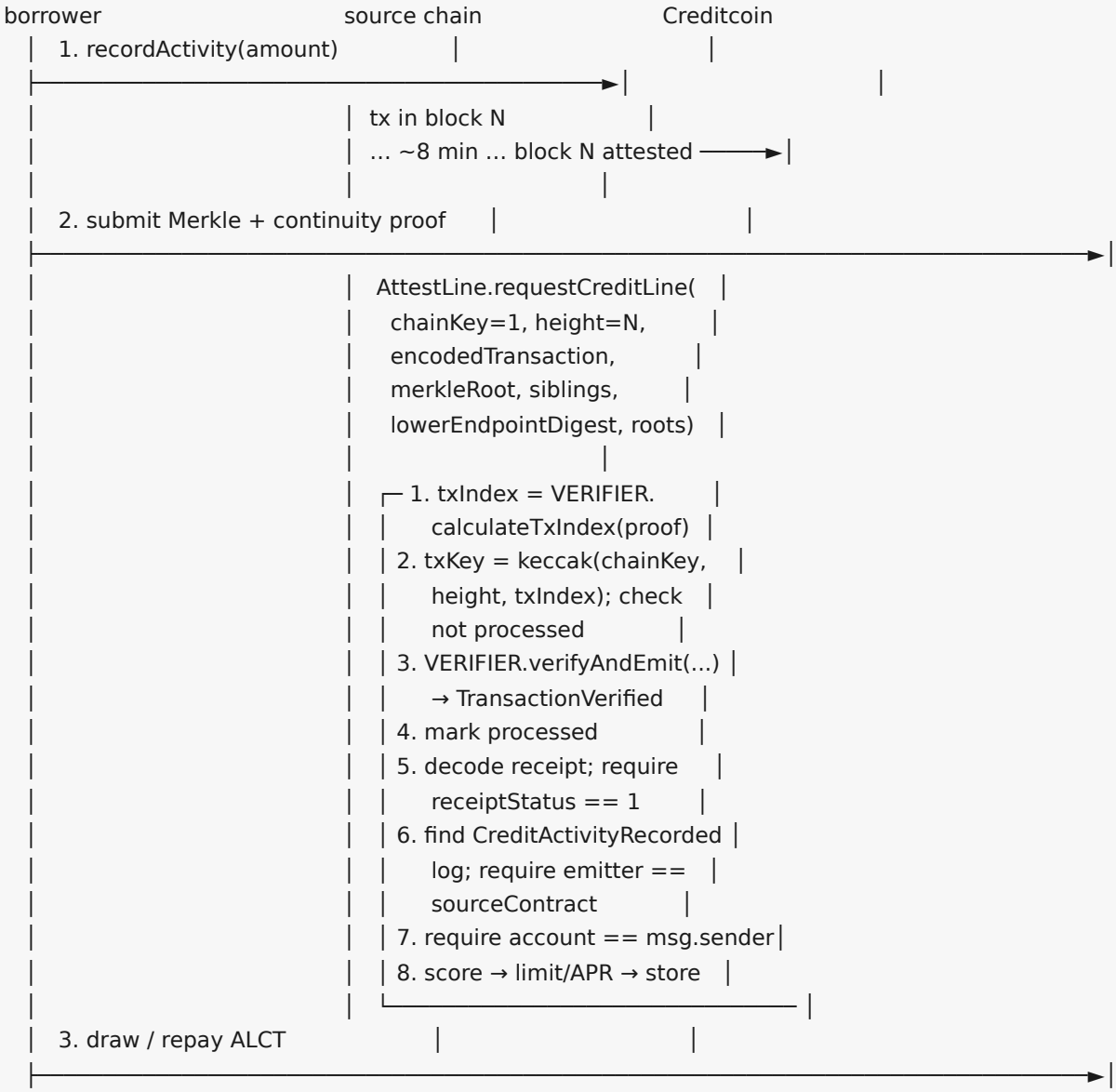
`NativeQueryVerifierLib.getVerifier()` (in `contracts/interfaces/INativeQueryVerifier.sol`) returns the Block Prover instance. The interface is vendored because the `@gluwa/usc-contracts` npm package ships a lean copy without `verifyAndEmit` / `calculateTxIndex`; struct layouts and signatures are byte-identical with the official example (`USCMinter` / `VerifierInterface.sol` in `gluwa/usc-testnet-bridge-examples`) and the canonical precompile ABI.

3. Chain keys

A **chain key** identifies a source chain on Creditcoin. `chainKey 1 = Ethereum Sepolia`. Supported chains, attested heights and continuity bounds are read from the ChainInfo precompile:

```
const info = new chainInfo.PrecompileChainInfoProvider(creditcoinProvider);
const chains = await info.getSupportedChains();        // [{ chainKey, chainId, chainName, chainEncoding }]
```

4. The proof flow, step by step



5. EvmV1 encoding (what the proof actually contains)

The proof builder (`proofProvider.service.ProofBuilder` in `@gluwa/usc-sdk` , backed by `https://prover.cc3-testnet.creditcoin.network`) returns a `ContinuityResponse` :

```
{
  chainKey, headerNumber, txIndex, txHash,
  txBytes,                          // EvmV1 ABI-encoded tx + receipt
  merkleProof:  { root, siblings: [{ hash, isLeft }] },
  continuityProof: { lowerEndpointDigest, roots: bytes32[] },
  cached, generatedAt
}
```

`txBytes` is `abi.encode(uint8 txType, bytes[] chunks)`. For a legacy (type 0) transaction:

- `chunks[0]` = common fields: `(uint64 nonce, uint64 gasLimit, address from, bool toIsNull, address to, uint256 value, bytes data)`
- `chunks[1]` = type-0 fields: `(uint128 gasPrice, uint256 v, bytes32 r, bytes32 s)`
- `chunks[2]` = receipt: `(uint8 receiptStatus, uint64 gasUsed, (address, bytes32[], bytes)[] logs, bytes bloom)`

`EvmV1Decoder` (from `@gluwa/usc-contracts`, linked at deploy time) decodes exactly this. The test fixtures in `test/fixtures/encoded-transactions.ts` reproduce the encoding byte-for-byte, and the client decodes the same format in TypeScript.

6. Why `receiptStatus` must be checked

The precompile proves **inclusion**: the transaction is in a finalized, attested block. It does **not** prove the transaction succeeded — a reverted transaction is still included in a block. An ASC that skipped the status check could be tricked by a proof of a *failed* transaction. AttestLine therefore always:

```
EvmV1Decoder.ReceiptFields memory receipt = EvmV1Decoder.decodeReceiptFields(encodedTransaction);
require(receipt.receiptStatus == 1, "AttestLine: transaction did not succeed");
```

(`TransactionDidNotSucceed` .)

7. Replay protection

- The transaction index is derived from the Merkle sibling path via `VERIFIER.calculateTxIndex`: walking the path leaf→root, `sibling[i].isLeft == true` sets bit `i` of the index.
- The replay key is `txKey = keccak256(chainKey, blockHeight, txIndex)` — the exact packed layout of the official ASCMinter example (chainKey as a full word, blockHeight in the top 8 bytes of the next word, txIndex in the following 8 bytes; 72 bytes hashed).
- `processedQueries[txKey] = true` after successful verification ⇒ the same proof can never be replayed (e.g. to re-grant a line, or to double-claim).

8. Event extraction & allowlisting

`getLogsByEventSignature(receipt, CREDIT_ACTIVITY_EVENT_SIGNATURE)` filters the receipt logs for `keccak256("CreditActivityRecorded(address,uint256)")`. The first match is processed and must:

- have been emitted by the owner-registered `sourceContract` (checked via the log's `address_` field — exactly like `USCLoanManager` checks its source loan contract),
- carry exactly two topics (signature + indexed account) and 32 bytes of data (`uint256 amount`),
- have its indexed `account` equal `msg.sender`.

9. SDK usage (TypeScript)

```
import { proofProvider, chainInfo } from "@gluwa/usc-sdk";

const proofBuilder = new proofProvider.service.ProofBuilder(chainKey, proofBuilderUrl);
await proofBuilder.waitUntilHeightAttested(chainKey, blockNumber); // polls 15s / 15m default
const result = await proofBuilder.getProof(txHash);              // { success, data, error }

const info = new chainInfo.PrecompileChainInfoProvider(ccProvider);
await info.getSupportedChains();
```

`AttestLineClient` in `src/client.ts` wraps this end to end (including fast pre-validation of the source receipt before the ~8-minute attestation wait).

10. Test strategy (deterministic, no RPC)

- `MockVerifier` implements the precompile interface with the same observable behavior (canonical Keccak Merkle verification, `calculateTxIndex`, strict (chainKey, height, root) enforcement, optional fail-all mode). Tests inject its bytecode at `0x...0FD2` with `hardhat_setCode`, so AttestLine calls the precompile address for real.
- Fixtures build EvmV1 encodings + Merkle proofs with the exact SDK/chain scheme, so the *same* bytes are used on-chain (MockVerifier recomputes the root) and in the client tests (TS decode).
- The matrix covers: happy path, replay protection, wrong chainKey/height/root, failed receipt, missing/foreign events, non-whitelisted emitter, account mismatch, limit/interest edge cases, defaults, and ownership.